Mini-IPHOREOS

From a Bayesian belief controller to a frozen, held-out evaluation
the Baseline system + four Extended-system milestones + the Transfer system

Florent Ouabo Kamkumo, PhD

4 October 2026



AI-generated concept illustration, not a photograph.

What this talk covers

1. What is Mini-IPHOREOS?
2. The Baseline system
3. The Extended system: a richer question
4. Milestone 1: fair baselines + a planner that doesn't stall
5. Milestone 2: context memory
6. Milestone 3: an explicit sensor-bias reference
7. Milestone 4: freeze it, then look at new data
8. The full picture
9. Toward a physical experiment
10. What to buy
11. The Transfer system: a cross-task test of O3

The task, in one picture



AI-generated concept illustration, not a photograph.

A small two-wheeled robot approaches a cardboard wall and must **stop with its sensor face 21–29 cm away** (nominal target 25 cm).

- Noisy ultrasonic distance readings
- Imperfect, pulse-controlled motion
- Must decide: move, measure, or stop and verify

Same observable task, from the first line of code to the last. Every milestone below changes *how the controller thinks*, never the task or the hardware interface.

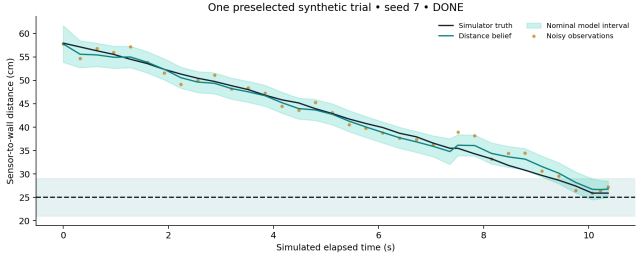
IPHOREOS: concept / event / context / policy

- **Concept**: a goal state ("sensor face within band of the wall")
- **Event**: something observed or realised (a reading, a movement outcome)
- **Context**: a hidden regime that shapes dynamics (surface speed, sensor bias)
- **Policy**: a short sequence of actions chosen to make concepts come true

What Mini-IPHOREOS actually is

A reduced, rule-based demonstration of this idea, a disciplined Bayesian belief controller, not a complete active-inference implementation. Two code lines: `baseline_v1` (the Baseline system) and `Mini_IPHOREOS_EXT1` (the Extended system), sharing the same sensor model and task.

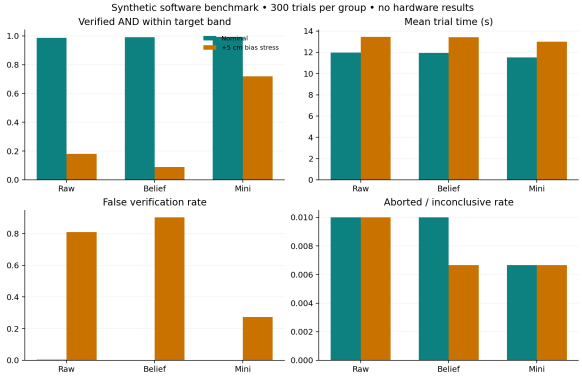
The Baseline system: a scalar Bayesian belief



- One state: distance to the wall
- Gaussian belief, updated from each noisy reading
- A one-step rule: move, measure, or verify and stop
- Three comparator policies: **Raw** (reading alone), **Belief** (Bayesian), **Mini** (Bayesian + verification band)

3,600 synthetic trials, computer-tested.
No physical robot has run this code.

The Baseline system's open problem: an unknown sensor bias



Under a **+5 cm constant sensor bias**, with no way to detect it:

- **Mini**'s verified-in-band rate falls from ≈0.99 to ≈0.72
- its false-verification rate rises from ≈0 to ≈0.28

A confident, narrow belief is not the same as an accurate one. This single finding motivates two of the Extended system's four milestones.

The Extended system: four milestones, one six-item plan

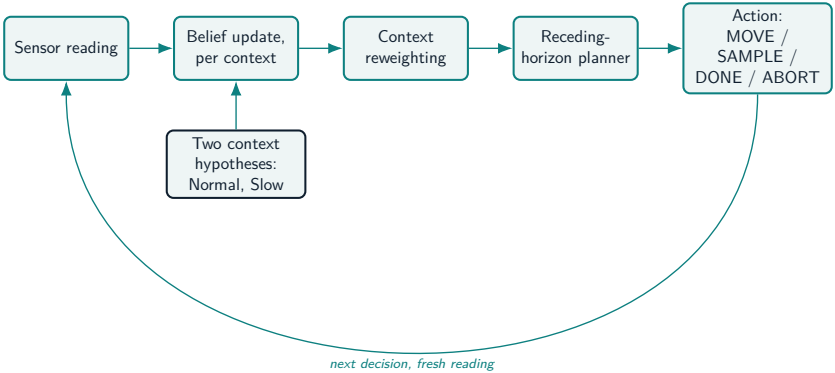
The Baseline system asks: *given uncertain distance, move, measure, or stop?*
The Extended system asks: *given uncertain distance **and speed**, under one of several possible contexts, what short sequence of actions best balances progress, cost, and the value of more information?*

The original six-item improvement plan

1. Fair, rule-based comparator baselines
2. A short-horizon (receding-horizon) planner
3. Context memory, so a recognised regime speeds up adaptation
4. A change-point detector, so memory is *fast*, not just present
5. An explicit sensor-bias reference
6. A frozen, held-out evaluation

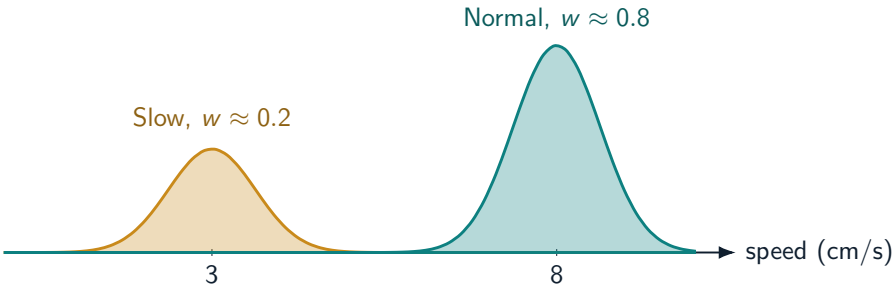
All six are now addressed, this talk walks through how.

The whole system, in one loop



One decision = one pass around this loop. Every milestone below changes one box or one arrow in it.

What "two movement contexts" looks like



Two running speed guesses held at once, sketch only, not to calibrated scale. As readings arrive, whichever guess predicts them better wins weight.

Milestone 1: a two-step planner, scored honestly



- Joint (distance, speed) belief, two-mode mixture
- Two-step receding-horizon planner with an explicit **no-progress cost**
- Fair comparator: `v1_fair`, a rule-based policy given the *same* context information as the Extended system, not a strawman

Without the no-progress cost, a one-step planner can prefer MEASURE forever once near the target. The two-step horizon fixes this directly.

Milestone 1: what it bought

- `v1_fair` roughly **halves** `v1`'s move count on the `normal` scenario (ratio ≈0.54), for the same success rate
- No-information policies (`adaptive_no_information`, `ext1_no_information`) stay at **1.00 success**: the historically riskiest combination (context-aware, no active information-seeking) no longer stalls or fails

Still open after this milestone

Context memory, a sensor-bias reference, and a held-out evaluation.

Milestone 2: a negative result, first

The hypothesis

Once a regime (normal / slow) has been seen before, *recognising* it again should let the controller adapt faster than re-estimating from scratch.

Attempt 1, a persistent library (failed)

Keep a running mean/variance per context, reuse it as a prior. Result: no measurable speed-up on the 2nd, 3rd, ... visit to a regime. Tracing individual decisions (not guessing) found why: the library only changes the speed *target*, not how fast the belief itself re-converges.

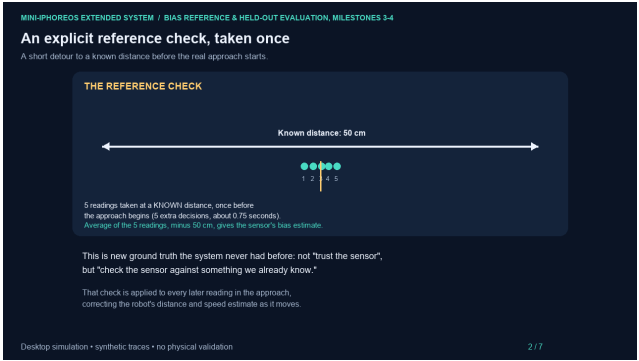
Milestone 2: recognise, then reuse



Attempt 2: a CUSUM change-point detector flags "this looks like a regime switch" from accumulated evidence, then **jumps** the belief directly to the stored value for that context.

- 1st visit to a regime: **identical** to no-memory ($\approx$4.3 decisions)
- 2nd+ visits: **3.2–3.3** vs **4.7–4.9** decisions to reconverge
- and more reliable, not just faster

Milestone 3: breaking the distance/bias confound



A constant sensor offset and a genuinely slower robot look **identical** from decisions alone: a confound the Baseline system's bias stress test exposed. **The fix**: 5 readings at a *known* 50 cm distance, taken once before the real approach. `mean(readings) - 50cm` becomes the bias estimate, fed into every later observation.

Milestone 3: what it recovered



Scenario	`ext1`	`ext1_bias_ref`
constant bias	0.86	**0.99**
random bias	0.53	**0.99**

Bias estimate error: **0.42 cm** on average.
Cost: 5 extra decisions, $\approx$0.75 s, paid once per trial.

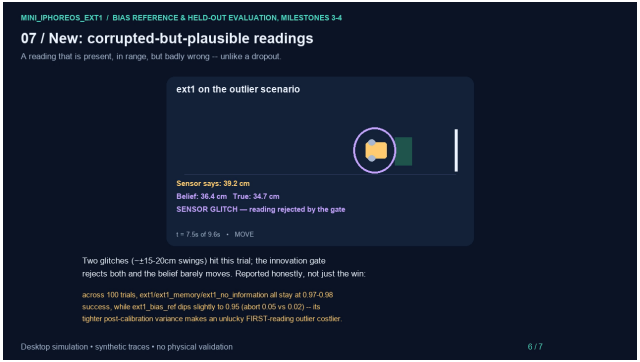
Milestone 4: the discipline

Why this milestone exists

Every number shown so far came from seeds 10000–10099, and several constants (CUSUM thresholds, library variances, jump weights, `REF_DISTANCE`, `N_REF`) were *selected by grid search on that same range*. That is development-set reporting, not confirmation.

- Every tunable constant frozen and written down **before** running anything further (`docs/FROZEN_PARAMETERS.md`)
- Full battery re-run, unmodified, on seeds **50000–50099**, never touched during three milestones of tuning
- Plus one new test: an **outlier** scenario (corrupted-but-plausible readings), distinct from `dropout`'s missing readings

Milestone 4: the new outlier scenario, reported honestly



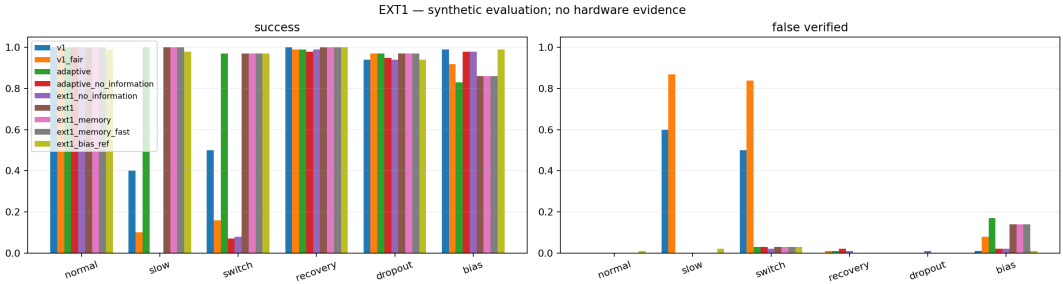
A reading that is *present* and in range, but badly wrong ($\pm$15–20 cm). The innovation-rejection gate catches it. Across 100 trials: most policies stay at **0.97–0.98** success. `ext1_bias_ref` dips to 0.95 (abort 0.05 vs 0.02), its tighter post-calibration variance makes an unlucky *first*-reading outlier costlier. Reported as a finding, not hidden.

Milestone 4: replication, dev seeds vs held-out seeds

Claim	Dev (10000–10099)	Held-out (50000–50099)
`v1_fair`/`v1` move ratio (normal)	0.54	0.54
No-information success	1.00	1.00
Memory+fast, 2nd+ visit (decisions)	3.2–3.3 vs 4.6–4.9	3.2–3.3 vs 4.6–4.9
Bias reference success (bias / random)	0.86/0.53 $\rightarrow$ 0.99	0.78/0.52 $\rightarrow$ 1.00

Every headline claim replicates on data never used to tune anything.

Nine policies, ten scenarios, one evaluation



All numbers from `results_ext1/summary.csv` and `results_ext1_heldout/`, produced by `experiments/run_simulation.py` and read (never asserted pass/fail) by `experiments/compare_heldout.py`.

Status

All six improvement-plan items are now addressed

Fair baselines, the short-horizon planner, context memory with fast recognition, an explicit bias reference, and a frozen held-out evaluation, each implemented, tested, and verified against held-out data.

What "status: addressed" does *not* mean

- Not a claim that the model is a complete active-inference implementation
- Not a claim that these numbers transfer to physical hardware
- **Physical validation remains entirely separate and has not been attempted**

What transfers directly, and what does not

- The sensor model, the motor-pulse mapping, and the task geometry: **shared** with the Baseline system's already-written physical tutorial (`baseline_v1/Tutorial_A_to_Z.pdf`)
- The Extended system's controller is plain NumPy, **incompatible** with MicroPython on the micro:bit as written
- Two routes: (a) rewrite the belief/planner in MicroPython-safe arithmetic, or (b) run the Extended system on a companion host, with the micro:bit as a serial sensor/motor bridge

Calibration that must be redone on hardware

Per-context speed calibration, *and* a physical version of the reference jig: a cut block or similar at a taped, ruler-verified 50 cm.

A staged test plan

1. Stationary reference-jig checks: does the bias estimate match a ruler?
2. Supported-wheel motor checks (wheels off the ground): pulse-to-distance calibration
3. Single-controller floor trials: does the Baseline system's own task still verify correctly?
4. Context-memory trials: alternate two physical surfaces/speeds, check reconvergence
5. Bias-reference trials: deliberately offset the sensor, check recovery
6. A frozen-parameter physical campaign: same constants as the desktop evaluation, no tuning on physical data

Full detail: `Tutorial_EXT1_EN.pdf`, Section 10.

What to buy: the complete ensemble

The Extended system needs nothing beyond the Baseline system's own build, plus a reference jig (a cut block or similar effectively free).

Item	Budget	Notes
Kitronik :MOVE Motor **5683**	€44.38	RS Germany listing incl. VAT
BBC micro:bit V2.21 Go Bundle	€18.90	BerryBase listing incl. VAT
Four matched alkaline AA cells	€5.00	Planning allowance
Ruler or tape measure	€5.00	Planning allowance
Removable floor tape	€4.00	Planning allowance
Broad cardboard target	€0.00	Reuse a packaging box
Laptop and USB data cable	€0.00	Existing equipment
Delivery across suppliers	€15.00	Checkout-specific
Price/replacement reserve	€15.00	Retained
Total planned envelope	**€107.28**	Unspent reserve remains yours

Buying it correctly

Buy exactly :MOVE Motor 5683

Not :MOVE mini, a Maqueen, or a Raspberry Pi Pico buggy, similar names do not imply the same motor driver or pin mapping.

- No purchase has been made as of this deck; prices must be reconfirmed at checkout
- Full sourcing links and alternative suppliers: `baseline_v1/Tutorial_A_to_Z.pdf`, chapter 03
- Assembly, wiring, and physical calibration: `baseline_v1/Tutorial_A_to_Z.pdf` directly, this project does not duplicate or change the Baseline system's motor programs

A later, separate question: does the library transfer to a new goal?

The question (Objective O3: "recompose shared representations for a new goal")

Milestone 2's library speeds up reuse *within* one task, across repeat visits. Does the *same learned library* transfer *across* tasks, a genuinely different goal?

Task B: a second approach, target 85 cm (band 81–89 cm), start 115–145 cm, same sensor, same motor model, same normal/slow regimes as Task A; only the goal differs.

Attempt 1, transfer the library as-is (failed)

Train on Task A, seed the learned library into Task B three ways (cold start / zero-update transfer / adapt with updates). Result: no measurable difference, any condition, any metric. Traced cause: the checkpoint learned speeds within noise of the plain calibration prior a cold-start belief already uses: Task A's training scenario draws from exactly those same calibration constants. Nothing to transfer, not a failure to transfer.

Attempt 2: give it something calibration can't already supply

Each of the 100 standard seeds becomes its own **deployment**: a persistent 0.6–0.85× speed bias, shared between that deployment's Task A training and Task B test, invisible to static calibration.

- Cold start now faces a provably wrong prior
- A checkpoint trained on that deployment's Task A should already know the correction

Metric	Cold vs. transfer
Success/moves/dec.-to-target	no diff.
Reconverge, post-switch	5.20 vs **3.79–3.85**

Paired by deployment (n=100): transfer reconverges faster in **61**, ties in 32, slower in only **7**.

Positive preliminary evidence for O3, with its actual scope

The library transfers across a genuine change of goal, and speeds up exactly what Milestone 2 showed it could ever speed up, *recognising which regime is active*, not overall task completion time. Both halves reported together, not the favourable one alone.

Where to go next

- `Tutorial_EXT1_EN.pdf`: the full LaTeX tutorial (Sections 1–11)
- `Mathematical_Model_EXT1.pdf`: every equation, every milestone, and the Transfer system
- Three captioned videos: Milestone 1, Milestone 2, Milestones 3–4 (`Mini_IPHOREOS_EXT1_Video/`)
- `baseline_v1/Tutorial_A_to_Z.pdf`: the physical build, start to finish

Thank you.